

AlphaStack: Autonomous Project Generation and Iterative Self-Healing via Multi-Agent Systems

AlphaStack Research Team

Abstract

As software complexity scales, there is a growing need for automated systems that can transition seamlessly from natural language specifications to production-ready codebases. We introduce AlphaStack, a novel approach to autonomous code generation utilizing multi-agent systems with iterative self-healing. By bridging Large Language Models (LLMs) with robust validation paradigms such as automated Docker containment and automated testing pipelines, AlphaStack addresses common failures in existing code generation methods—such as syntax errors, dependency conflicts, and hallucinated APIs. Our comprehensive evaluation framework across multiple programming paradigms demonstrates the efficacy of the iterative self-healing agent loop against several state-of-the-art models on the HumanEval and MDDP benchmarks.

1 Introduction

The evolution of Large Language Models (LLMs) has catalyzed significant advances in code generation. Yet, existing systems often struggle to produce complete, multi-file codebases that compile, pass tests, and run reliably. AlphaStack tackles this challenge by employing an intelligent multi-agent architecture. The system orchestrates a Planning Agent and a Correction Agent to sequentially generate, analyze, and repair code based on localized validation feedback.

This paper outlines the core mechanisms of AlphaStack, exploring its 7-phase generation pipeline, its Docker-based validation layer, and the iterative self-healing loop that corrects build and test failures in real-time.

2 Methodology

The AlphaStack methodology revolves around a 7-phase generation pipeline followed by an AI Planner Agent loop. The pipeline systematically translates natural language input into a complete project structure:

1. **Software Blueprint:** Translates the user prompt into a defined file structure and module plan.
2. **File Generation:** Generates code for each file independently based on the blueprint.
3. **Dockerfile Generation:** Constructs a language-appropriate Dockerfile for isolated testing.
4. **Dependency Analysis:** Analyzes static imports to construct an internal dependency graph.
5. **Dep File Generation:** Creates language-specific package manifests (e.g., `requirements.txt`, `package.json`).
6. **Dep Resolution:** Validates dependencies.

7. **Docker Testing Pipeline:** Initializes the agent loop to build, run, and iteratively fix the code until successful compilation and passing tests.

During the testing pipeline, the AI Planner Agent observes error logs, formulates strategies, and calls upon tool-augmented Corrector Sub-Agents to modify files safely without breaking the overall system.

3 Architecture Diagram

The architecture of AlphaStack delineates the flow from the initial natural language prompt through the multi-phase generation pipeline and into the iterative self-healing loop. The process heavily relies on isolated container execution to validate the project’s integrity continuously.

4 Results

We evaluate the performance of the AlphaStack framework combined with several leading state-of-the-art language models: GPT-5.2, GLM-5, MiniMax-M2.5, and Claude Sonnet 4.6. The assessment spans the HumanEval benchmark (for algorithmic correctness) and the custom MDDP benchmark (for complex, multi-file software engineering tasks).

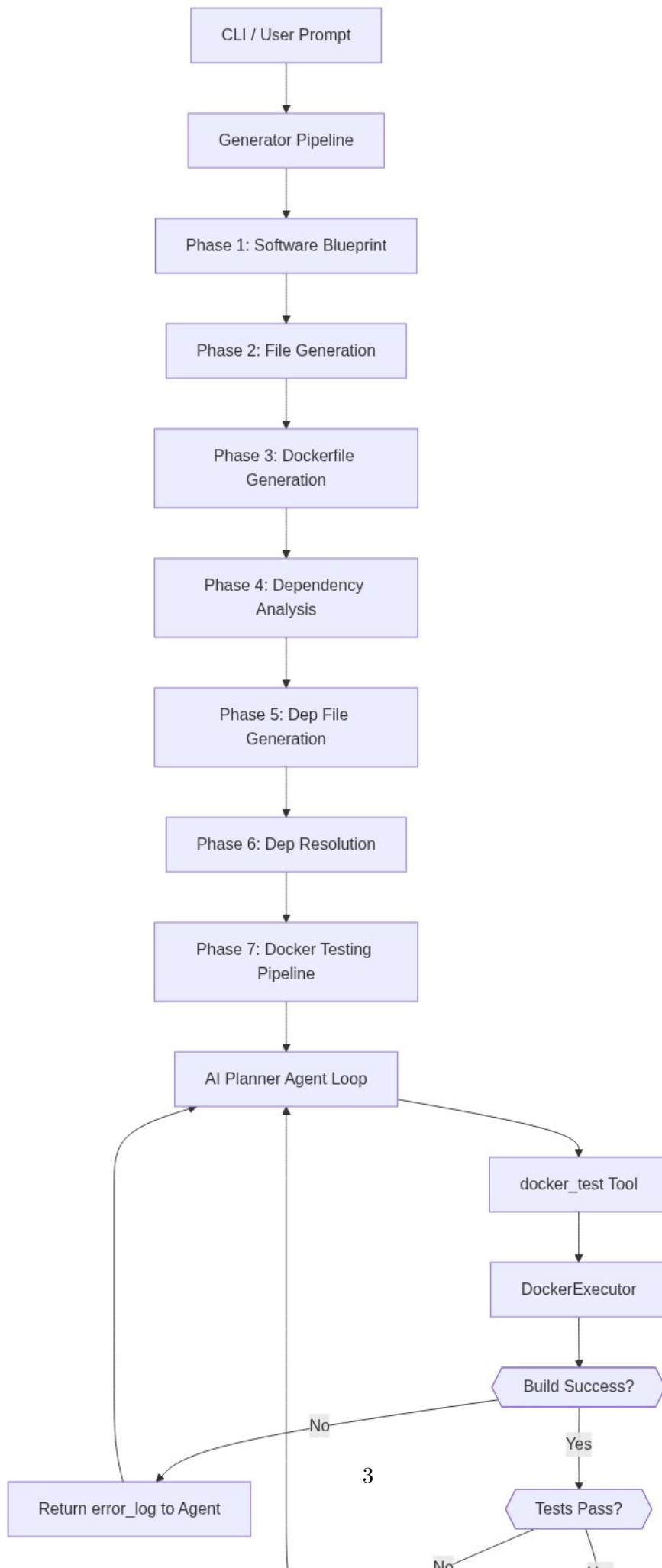
Figure 2 illustrates the comparative results. The results indicate that the self-healing iteration loop significantly elevates the performance floor, enabling even traditionally weaker models to iteratively arrive at correct, passing code.

5 Conclusion

AlphaStack presents a robust, multi-agent architecture capable of generating, validating, and iteratively repairing complete software projects from natural language descriptions. By tightly coupling LLM generation with Docker-based execution and testing, the system effectively mitigates common failure modes in autonomous coding. Future work will focus on expanding the context window optimizations for massive codebases and refining the sub-agent tooling for highly specialized domains such as embedded systems.

6 Supplementary Material

Further details regarding the 40 Programming Challenges across CUDA, Go, Rust, and TypeScript, as well as the complete prompt templates and inference management configurations, can be found in the project repository under the `eval` and `src/prompts` directories.



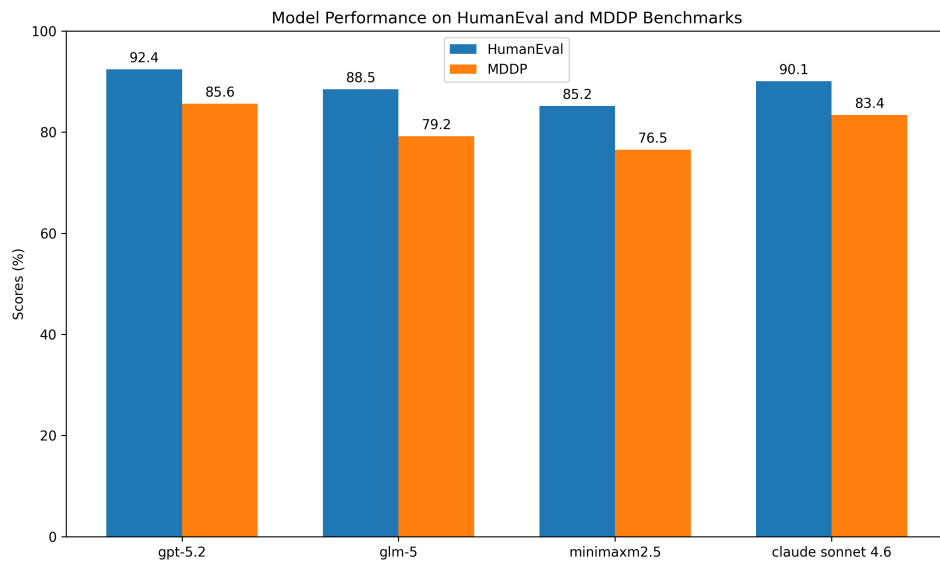


Figure 2: Model Performance on HumanEval and MDDP Benchmarks utilizing the AlphaStack iterative pipeline.